Trabajo Práctico 6

Inciso 1

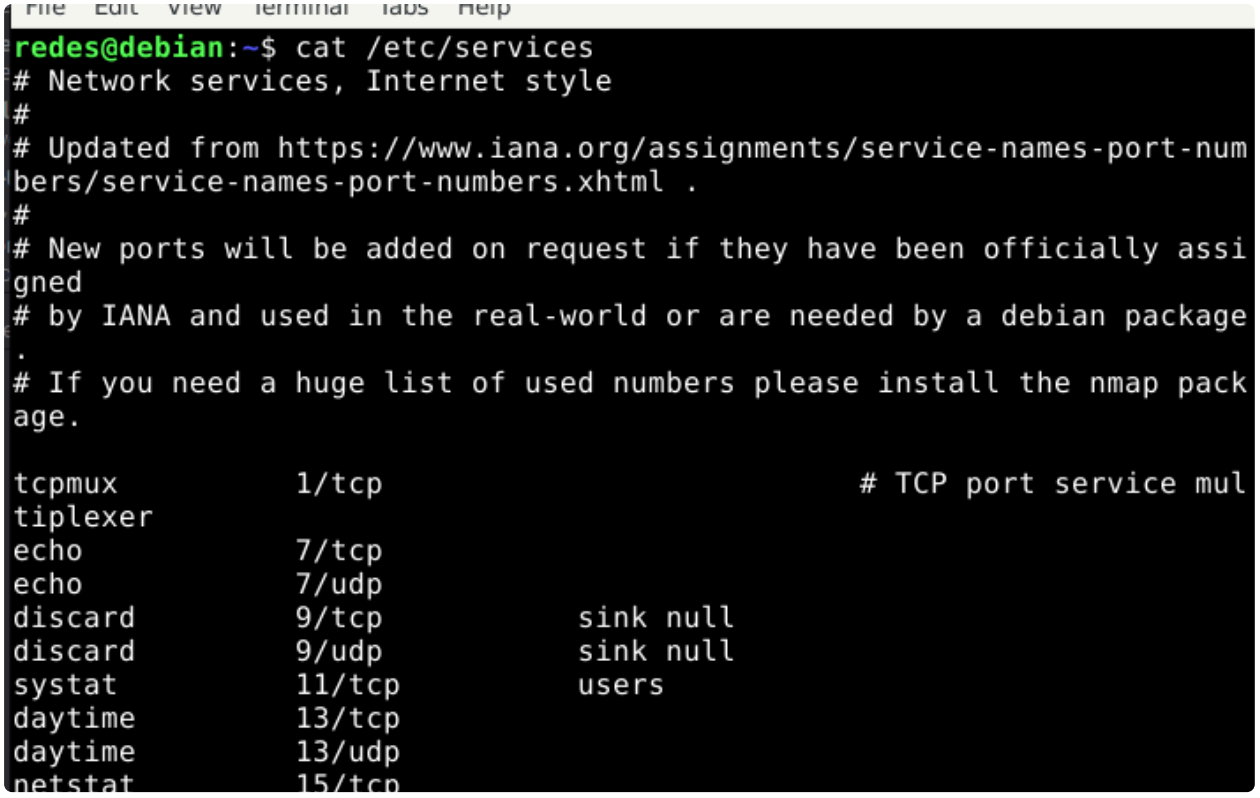
¿Cuál es el puerto por defecto que se utiliza en los siguientes servicios?
Web / SSH / DNS / Web Seguro / POP3 / IMAP / SMTP
Investigue en qué lugar en Linux y en Windows está descrita la asociación utilizada por defecto para cada servicio

Respuesta:

Servicio	**Protocolo**	**Puerto**
Web	TCP	80
SSH	TCP	22
DNS	UDP/TCP	53
Web seguro	TCP (UDP en HTTP/3 con quic)	443
POP3	TCP	110
IMAP	TCP	143
SMTP	TCP	25 (587 para submission/msa)

- **En Linux:** La asociación estandarizada entre los nombres de los servicios, sus números de puerto y el protocolo de transporte correspondiente (TCP/UDP) está descrita en el archivo **/etc/services**. Herramientas de red como `ss` o `netstat` leen este archivo para traducir los números de puerto a nombres de servicios legibles. Se puede consultar o filtrar mediante órdenes como `grep` (por ejemplo: grep domain /etc/services).
- **En Windows:** La tabla de correspondencias equivalente se encuentra ubicada en el archivo de texto del sistema **C:\Windows\System32\drivers\etc\services** (el cual se puede

inspeccionar con cualquier editor de texto como el Bloc de notas).



Inciso 2

Investigue qué es multicast. ¿Sobre cuál de los protocolos de capa de transporte
funciona? ¿Se podría adaptar para que funcione sobre el otro protocolo de capade transporte? ¿Por qué?

Multicast cisco

Respuesta

Multicast es un esquema de comunicación de red en el cual un único emisor transmite paquetes de datos a un **grupo de receptores interesados** de forma simultánea a través de una única operación de envío.

A diferencia de:

- **Unicast (Unidifusión):** Comunicación de un emisor a un único receptor (punto a punto).
- **Broadcast (Difusión):** Comunicación de un emisor a *todos* los hosts de la red local.

En **multicast**, los paquetes se envían a una **dirección IP de grupo de multidifusión** (en IPv4, direcciones de Clase D en el rango `224.0.0.0` a

`239.255.255.255`). La infraestructura de red (routers y switches) se encarga de replicar los paquetes únicamente en los enlaces que conducen a hosts que se hayan unido explícitamente a dicho grupo.

Este mismo funciona sobre el protocolo de **UDP**. Al ser este sin conexión y no fiable, UDP empaqueta los datos en datagramas y los entrega a la capa de red con la dirección multicast destino.

¿Se podría adaptar para que funcione sobre otro? Ejemplo TCP

La respuesta es no, porque:

- **TCP es estrictamente punto a punto (orientado a conexión):** Una conexión TCP existe de manera exclusiva entre un único emisor y un único receptor
- **Demultiplexación por 4-upla:** Un socket de conexión TCP en el sistema operativo queda determinado unívocamente por una **4-upla**: `(IP origen, puerto origen, IP destino, puerto destino)`. Esto exige un estado de sesión individualizado por cada par cliente-servidor, lo cual es incompatible conceptualmente con enviar un solo segmento hacia un grupo indeterminado de receptores.
- **Implosión de reconocimientos (ACK Storm / ACK Implosion):** TCP garantiza la entrega fiable mediante reconocimientos (*ACKs*) explícitos/acumulativos y temporizadores de retransmisión. Si un emisor intentara enviar tráfico TCP en multicast a cientos o miles de receptores simultáneamente, recibiría de golpe una gran cantidad de ACKs que colapsaría tanto al servidor emisor como a los enlaces de red.
- **Imposibilidad de gestionar el control de flujo y de congestión:** TCP ajusta dinámicamente su velocidad de transmisión según la capacidad del receptor (**ventana de recepción**) y el estado de la red (**ventana de congestión**). En un grupo multicast, cada receptor tiene diferentes capacidades de buffer y atraviesa enlaces con distintos niveles de congestión o retardo; por ende, no existe una única tasa de envío o tamaño de ventana válido para todo el grupo

Inciso 3

Investigue cómo funciona el protocolo de aplicación FTP teniendo en cuenta las

diferencias en su funcionamiento cuando se utiliza el modo activo de cuando se utiliza el modo pasivo ¿En qué se diferencian estos tipos de comunicaciones del resto de los protocolos de aplicación vistos?

Respuesta:

El protocolo de aplicación **FTP (File Transfer Protocol)**, definido en la RFC 959, se utiliza para la transferencia de archivos entre un cliente y un servidor a través del protocolo de transporte **TCP**.
A diferencia de la mayoría de los protocolos de aplicación, FTP se caracteriza por utilizar **dos canales de conexión TCP independientes** (arquitectura fuera de banda o *out-of-band*):

1. **Canal de control (Puerto TCP 21 por defecto):** Se establece al inicio de la sesión y permanece abierto durante toda la interacción para enviar comandos (autenticación, listado de directorios, comandos de transferencia) y recibir códigos de respuesta.
2. **Canal de datos:** Se abre de manera dinámica únicamente cuando se solicita transferir un archivo o listar un directorio, y se cierra al finalizar dicha transferencia.

La forma en que se establece el canal de datos determina si se opera en **Modo Activo** o **Modo Pasivo**.

Modo activo:
En el modo activo, el establecimiento del canal de datos es **iniciado por el servidor** hacia el cliente:

1. **Conexión de Control:** El cliente abre una conexión TCP desde un puerto efímero local hacia el puerto **TCP 21** del servidor.
2. **Solicitud de Datos:** Cuando el cliente desea transferir un archivo, abre un segundo puerto efímero local en su máquina y envía al servidor el comando `PORT IP_cliente,puerto_M` a través del canal de control.
3. **Conexión de Datos:** El servidor recibe el comando e inicia una nueva conexión TCP desde su puerto reservado TCP 20 hacia la dirección IP del cliente y su puerto
4. **Inconveniente:** Si el cliente se encuentra detrás de un firewall o un router con traducción de direcciones (NAT), la solicitud de conexión entrante

iniciada por el servidor hacia el puerto del cliente será rechazada o bloqueada por seguridad.

Modo pasivo:
El modo pasivo fue diseñado para solucionar los problemas de conectividad del modo activo, haciendo que **ambas conexiones sean iniciadas por el cliente**:

1. **Conexión de Control:** El cliente se conecta desde un puerto efímero al puerto **TCP 21** del servidor.
2. **Solicitud de Modo Pasivo:** Cuando se requiere transferir datos, el cliente envía el comando `PASV` sobre el canal de control.
3. **Respuesta del Servidor:** El servidor abre un puerto efímero aleatorio en su propia máquina y le responde al cliente con el mensaje `227 Entering Passive Mode (IP_servidor,puerto_P)`.
4. **Conexión de Datos:** El cliente **inicia la conexión TCP** desde su puerto efímero $+ 1$ hacia la IP del servidor y el puerto efímero que este le indicó.
5. **Ventaja:** Dado que ambas conexiones TCP se originan desde el cliente hacia el servidor, los cortafuegos y routers NAT del lado del cliente permiten el tráfico saliente sin inconvenientes.

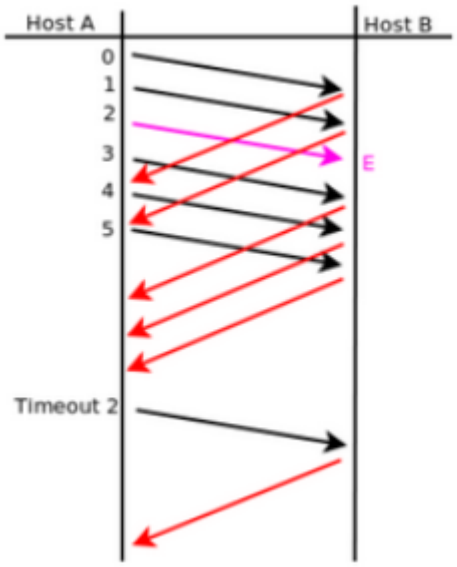
El funcionamiento de FTP se diferencia sustancialmente de otros protocolos de aplicación vistos (como HTTP, SMTP, POP3, IMAP, SSH o DNS) en dos aspectos fundamentales:

- **Separación en dos canales (Control fuera de banda / Out-of-band):**
  - Protocolos como **HTTP**, **SMTP** o **SSH** utilizan una **única conexión TCP** por la cual se transmiten de forma entrelazada (*in-band*) tanto las peticiones de control como el contenido de los datos.
  - **FTP**, en cambio, mantiene el canal de control separado del canal de transporte de datos. Los archivos nunca viajan por el puerto 21.
- **Dirección de inicio de la conexión TCP:**
  - En protocolos convencionales cliente-servidor (HTTP, HTTPS, SSH, DNS), **el cliente es siempre quien inicia todas las conexiones TCP** hacia los puertos bien conocidos del servidor.

- En **FTP Modo Activo**, se invierte este rol para el canal de datos: **el servidor es quien inicia la conexión de retorno** hacia el puerto efímero expuesto por el cliente.

Inciso 4

Suponiendo Selective Repeat; tamaño de ventana 4 y sabiendo que E indica que el mensaje llegó con errores. Indique en el siguiente gráfico, la numeración de los ACK que el host B envía al Host A.



Selective-repeat apunte
ACK selective repeat

Respuesta:

Siguiendo las flechas negras (paquetes de datos enviados por A) y las rojas (ACKs enviados por B de abajo hacia arriba), la secuencia de ACKs que el Host B emite es la siguiente:

1. **Llega el Paquete 0 (bien):** Host B responde enviando **ACK 0**.
2. **Llega el Paquete 1 (bien):** Host B responde enviando **ACK 1**.
3. **Llega el Paquete 2 con error ():** Host B lo descarta $\rightarrow$ **No se envía ACK** para el 2 en este momento.
4. **Llega el Paquete 3 (bien, pero desordenado):** Como está dentro de la ventana de recepción ($\mathrel{\dot{-}} 4$), B lo guarda en el búfer y responde enviando **ACK 3**.
5. **Llega el Paquete 4 (bien, desordenado):** B lo guarda en el búfer y responde enviando **ACK 4**.

6. **Llega el Paquete 5 (bien, desordenado):** B lo guarda en el búfer y responde enviando **ACK 5**.
7. **Timeout del Paquete 2:** El Host A, al no recibir el ACK de 2, retransmite únicamente el paquete 2.
8. **Llega el Paquete 2 retransmitido (bien):** El Host B lo recibe, lo junta con los paquetes 3, 4 y 5 que tenía en el búfer, entrega todo el bloque ordenado a la capa superior y responde enviando **ACK 2**.

Inciso 5

¿Qué restricción existe sobre el tamaño de ventanas en el protocolo Selective Repeat?

Respuesta:

En el protocolo **Repetición Selectiva** (*Selective Repeat* - SR), la restricción fundamental es que el tamaño de la ventana () debe ser menor o igual a la mitad del tamaño del espacio de números de secuencia disponibl:

$$\text{Tamaño de la ventana } () \leq \frac{\text{Tamaño del espacio de números de secuencia}}{2}$$

Si los números de secuencia se representan mediante un campo de bits en la cabecera (lo que genera un espacio total de 2^n números de secuencia), la restricción se expresa formalmente como:

$$\leq \frac{2^n}{2}$$

¿Por qué existe esta restricción?

A diferencia de protocolos como *Go-Back-N*, en Repetición Selectiva el emisor y el receptor no siempre tienen una visión idéntica de qué paquetes se han entregado correctamente, por lo que sus ventanas no avanzan exactamente al mismo tiempo y el receptor almacena en un *buffer* los paquetes fuera de orden.

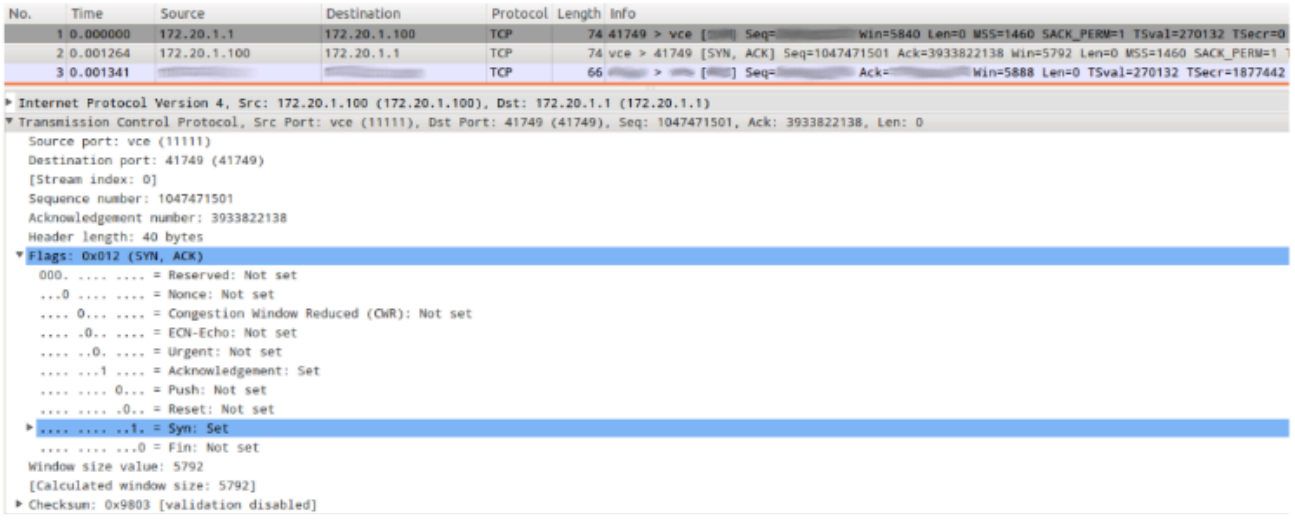
Si el tamaño de la ventana supera la mitad del espacio de números de secuencia, surge una ambigüedad insalvable para el receptor, que no puede distinguir si un paquete entrante es una **retransmisión de un paquete antiguo** o la **primera llegada de un paquete nuevo**:

1. **Escenario A (Pérdida de reconocimientos):** El emisor transmite los paquetes de la ventana actual y el receptor los acepta enviando sus respectivos ACKs; sin embargo, todos los ACKs se pierden en el canal4. Al vencer el temporizador, el emisor retransmite el primer paquete de la ventana.
2. **Escenario B (Transmisión y avance exitoso):** Los paquetes y sus ACKs llegan correctamente, por lo que la ventana del receptor avanza. El emisor transmite los siguientes paquetes del flujo, reutilizando los números de secuencia iniciales del ciclo previo.

Si la ventana es demasiado grande ($\geq 2^{n-1}$), el receptor observa llegar un paquete con el mismo número de secuencia en ambos escenarios y no tiene forma de saber si debe descartarlo por ser duplicado o entregarlo a la capa superior como datos nuevos.

Inciso 6

De acuerdo a la captura TCP de la siguiente figura, indique los valores de los campos borroneados.



Three way handshake tcp

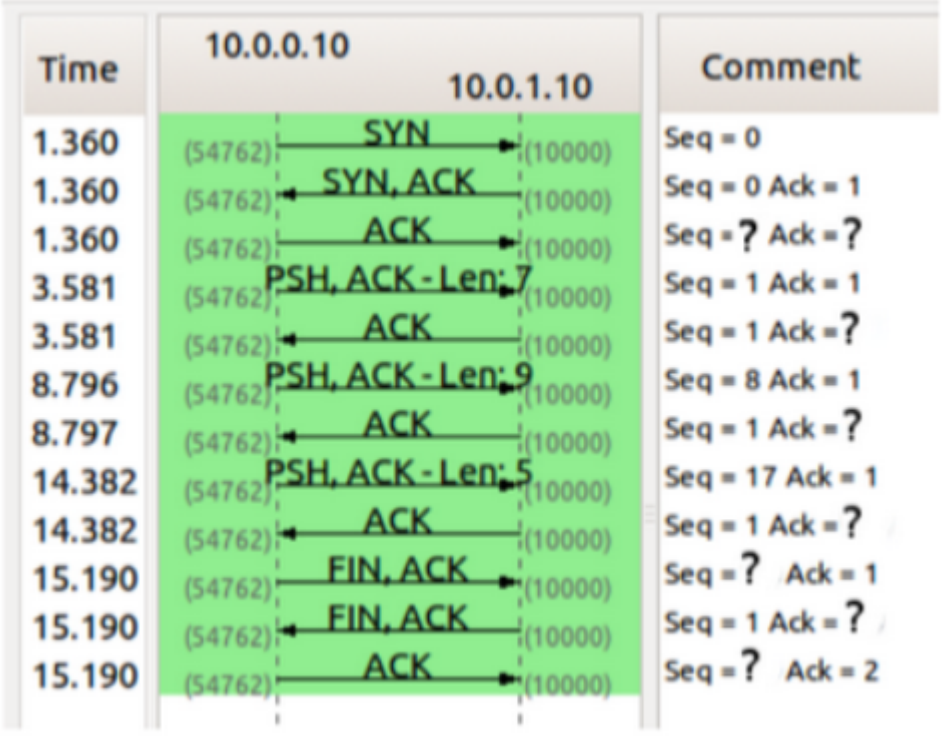
Respuesta:

- **Paquete 1 (Cliente → Servidor):**
  - **Tipo:** Segmento **SYN** (Inicio de conexión por parte del cliente).
  - **Seq:** (Valor borroneado del cliente, 3933822137 o un número base).

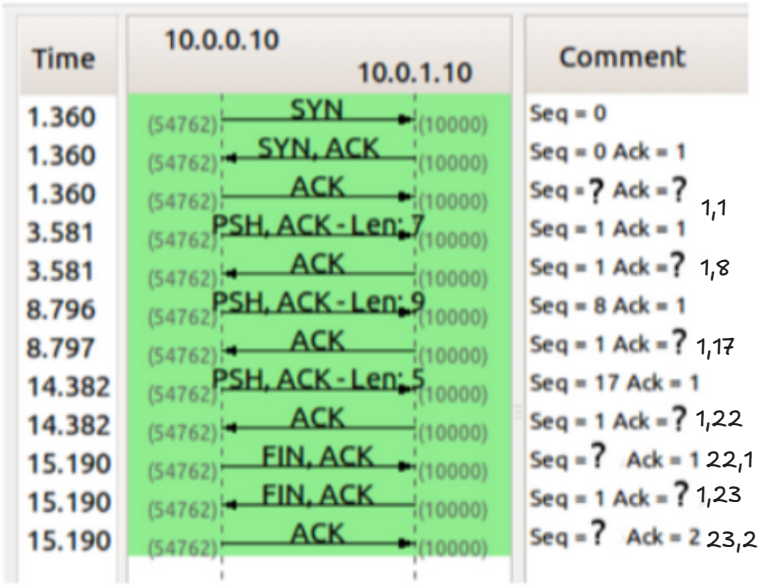
- **Paquete 2 (Servidor → Cliente):**
  - **Tipo:** Segmento **SYN, ACK**.
  - **Seq:** Corresponde al número de secuencia inicial elegido por el servidor (ej. `1047471501`).
  - **Ack:** Como confirma el SYN del cliente, su valor es el **Seq del cliente + 1** (es decir, $+ 1$).
- **Paquete 3 (Cliente → Servidor):**
  - **Tipo:** Segmento **ACK** (Tercer paso del handshake).
  - **Seq:** Debe continuar el flujo del cliente, por lo que su valor es el Seq inicial del cliente más los datos enviados en el primer segmento (como el SYN consume 1 número virtual de secuencia, el Seq pasa a ser $+ 1$).
  - **Ack:** Como confirma el SYN-ACK del servidor, el valor del Ack es el **Seq del servidor + 1** (es decir, `1047471501 + 1 = 1047471502`).

Inciso 7

Dada la sesión TCP de la figura, completar los valores marcados con un signo de
interrogación.



Respuesta



Nota de ayuda

$$\text{Siguiente Seq} = \text{Seq actual} + \text{Len (longitud de datos del segmento)}$$

$$\text{Ack a enviar} = \text{Seq recibido} + \text{Len del segmento recibido}$$

Inciso 8

¿Qué es el RTT y cómo se calcula? Investigue la opción TCP timestamp y los campos TSval y TSecr.

Respuesta:

El **RTT (Round-Trip Time o Tiempo de Ida y Vuelta)** es la cantidad de tiempo que transcurre desde que un emisor envía un paquete de datos (segmento TCP) hasta que recibe el mensaje de confirmación (*ACK*) correspondiente enviado por el receptor.
Dado que las condiciones de tráfico en la red varían continuamente, TCP necesita medir el RTT para ajustar de forma adaptativa el **temporizador de retransmisión (Timeout)**. Si el temporizador es demasiado corto, se producen retransmisiones innecesarias; si es demasiado largo, el protocolo reacciona muy lento ante pérdidas.

El cálculo estándar consta de cuatro pasos:

1. **Muestreo (SampleRTT):** Es la medición directa del tiempo transcurrido desde que se pasa un segmento a la capa IP hasta que se recibe su

_ACK_45.

- **Algoritmo de Karn:** TCP **nunca** calcula la muestra de RTT sobre segmentos que hayan sido retransmitidos, ya que no es posible saber si el *ACK* recibido responde a la primera transmisión o a una posterior (evitando lecturas erróneas).

2. **Procesamiento de la media ($\mathrm{EstimatedRTT}$):** Para suavizar las fluctuaciones puntuales, TCP calcula un promedio móvil ponderado exponencialmente (EWMA) asignando mayor peso a las muestras más recientes7:

$$\mathrm{EstimatedRTT} = (1 - \alpha) \cdot \mathrm{EstimatedRTT} + \alpha \cdot \mathrm{SampleRTT}$$

(El valor recomendado por la RFC 6298 para α es $0{,}125$ o $1/8$_).

3. **Cálculo de la variabilidad (DevRTT):** Estima cuánto se desvía habitualmente la muestra respecto al promedio:

$$\mathrm{DevRTT} = (1 - \beta) \cdot \mathrm{DevRTT} + \beta \cdot |\mathrm{SampleRTT} - \mathrm{EstimatedRTT}|$$

(El valor recomendado para β es $0{,}25$ o $1/4$_).

4. **Intervalo del temporizador ($\mathrm{TimeoutInterval}$):** Se define sumando al RTT estimado un margen de seguridad proporcional a su desviación:

$$\mathrm{TimeoutInterval} = \mathrm{EstimatedRTT} + 4 \cdot \mathrm{DevRTT}$$

La opción **TCP Timestamp** es una extensión de la cabecera TCP (definida originalmente en la RFC 1323 y actualizada en la RFC 7323) que ocupa 10 bytes dentro del campo de opciones de TCP.

Su activación resuelve dos problemas fundamentales en redes modernas13:

- Medición continua del RTT (*RTTM - Round-Trip Time Measurement*):Permite calcular el RTT en cada segmento enviado sin depender de temporizadores complejos por paquete.
- Protección contra reciclado de números de secuencia (*PAWS - Protection Against Wrapped Sequences*): En enlaces de muy alta velocidad, los números de secuencia de 32 bits vuelven a cero en cuestión de segundos. Las marcas de tiempo permiten diferenciar paquetes antiguos demorados de paquetes nuevos que reutilizan la misma secuencia.

Campos `TSval` y `TSecr`:

- **TSval (**Timestamp Value**):** Es el valor numérico del reloj del host emisor en el momento exacto en que transmite el segmento.
- **TSecr (**Timestamp Echo Reply**):** Es la "eco-respuesta". El host receptor toma el `TSval` del segmento que acaba de recibir y lo devuelve intacto dentro del campo `TSecr` de su segmento *ACK*.

¿Cómo se calcula el RTT con Timestamps?

Cuando el emisor original recibe la respuesta *ACK*, no necesita guardar registros de envío previos. Simplemente consulta su reloj interno actual y realiza una resta directa:

$$\text{SampleRTT} = \text{Hora}_{\text{Actual}} - \text{TSecr}$$

De esta manera, la opción de marcas de tiempo permite obtener mediciones instantáneas de RTT para múltiples paquetes simultáneamente y elimina la ambigüedad en presencia de retransmisiones.

Inciso 9

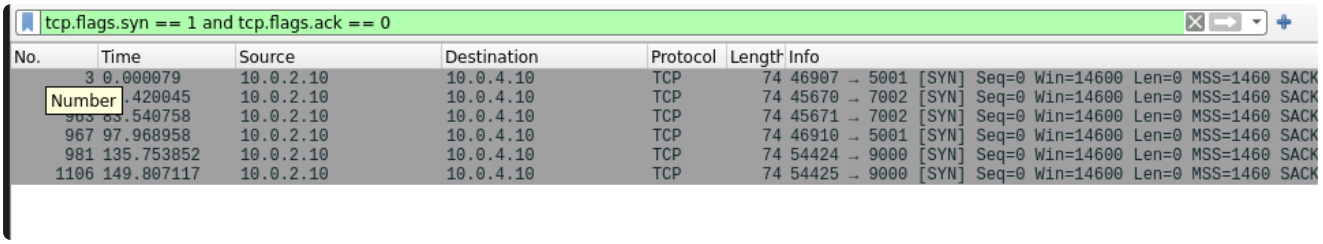
Para la captura tcp-captura.pcap, responder las siguientes preguntas.

1. ¿Cuántos intentos de conexiones TCP hay?
2. ¿Cuáles son la fuente y el destino (IP:port) para c/u?
3. ¿Cuántas conexiones TCP exitosas hay en la captura? ¿Cómo diferencia las exitosas de las que no lo son? ¿Cuáles flags encuentra en cada una?
4. Dada la primera conexión exitosa responder:
    1. Quién inicia la conexión?
    2. ¿Quién es el servidor y quién el cliente?
    3. ¿En qué segmentos se ve el 3-way handshake?
    4. ¿Cuáles ISNs se intercambian?
    5. ¿Cuál MSS se negoció?
    6. ¿Cuál de los dos hosts envía la mayor cantidad de datos (IP:port)?
5. Identificar primer segmento de datos (origen, destino, tiempo, número de fila y número de secuencia TCP).
    1. ¿Cuántos datos lleva?

2. ¿Cuándo es confirmado (tiempo, número de fila y número de secuencia TCP)?
3. La confirmación, ¿qué cantidad de bytes confirma?

6. ¿Quién inicia el cierre de la conexión? ¿Qué flags se utilizan? ¿En cuáles segmentos se ve (tiempo, número de fila y número de secuencia TCP)?

Inciso 9.1

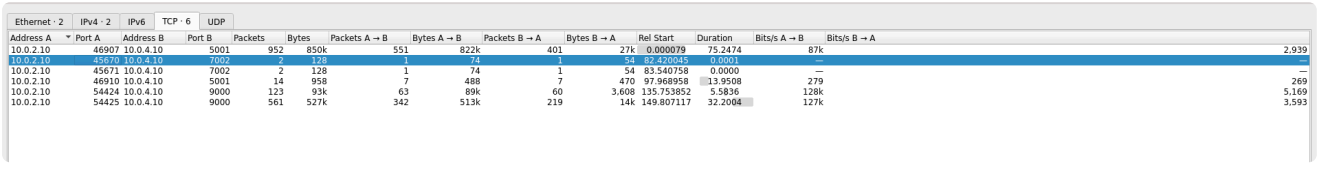


Hubo exactamente 6 intentos de conexión TCP

Inciso 9.2

Source	Destination
10.0.2.10:46907	10.0.4.10:5001
10.0.2.10:45670	10.0.4.10:7002
10.0.2.10:45671	10.0.4.10:7002
10.0.2.10:46910	10.0.4.10:5001
10.0.2.10:54424	10.0.4.10:9000
10.0.2.10:54425	10.0.4.10:9000

Inciso 9.3



1. ¿Cuántas conexiones TCP exitosas hay en la captura?
   - **Cantidad:** Hay **6 conexiones** TCP registradas en total en esa ventana de conversaciones (las 6 filas que ves en la lista).
   - Todas ellas llegaron a establecerse con éxito, ya que completaron el saludo inicial (*Three-way Handshake*) y acumularon intercambio de

paquetes y bytes en ambas direcciones (`Packets A -> B` y `Packets B -> A`).

2. ¿Cómo se diferencian las exitosas de las que no lo son?
   - **Conexión Exitosa:** Pasan por el proceso de 3 pasos (`SYN` → `SYN-ACK` → `ACK`), alcanzan el estado `ESTABLISHED` (o "E") y permiten la transferencia bidireccional de datos.
   - **Conexión No Exitosa (Fallida):** Son aquellas que **nunca llegan a establecerse**. Se identifican porque:
     - El cliente envía múltiples paquetes `SYN` repetidos (retransmisiones) sin recibir jamás una respuesta del servidor (host apagado o bloqueado por firewall).
     - El servidor responde de inmediato con un paquete de reinicio `RST, ACK`, indicando que el puerto está cerrado y la conexión es rechazada (*Connection refused*).
3. ¿Cuáles flags se encuentran en cada una
   - **En conexiones exitosas:**
     - **Establecimiento (*Handshake*):** Se usan los flags `SYN` (cliente), `SYN, ACK` (servidor) y `ACK` (cliente).
     - **Transferencia de datos:** Se usa principalmente la combinación `PSH, ACK` para enviar datos de aplicación y `ACK` para confirmarlos.
     - **Cierre:** Se utiliza la combinación `FIN, ACK` por parte de los hosts que inician el cierre de la sesión.
   - **En conexiones fallidas:**
     - Si el puerto está cerrado, encontrarás el flag `RST` (o la combinación `RST, ACK`).
     - Si hay pérdida de conexión o caídas por *timeout*, solo verás paquetes con el flag `SYN` repetidos constantemente sin ningún `SYN-ACK` de vuelta.

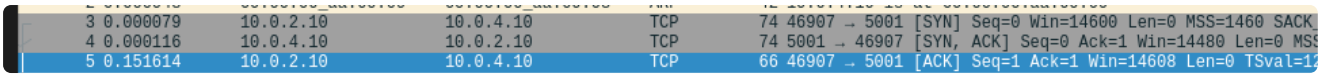
Inciso 9.4

La conexiópn es iniciada por 10.0.2.10:46907.

Servidor y cliente:

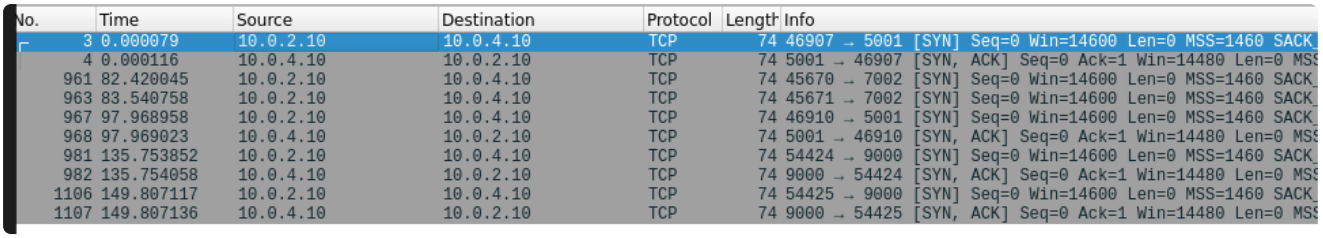
- Servidor: 10.0.4.10:5001.

- Cliente: 10.0.2.10:46907



3	0.000079	10.0.2.10	10.0.4.10	TCP	74 46907 → 5001 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
4	0.000116	10.0.4.10	10.0.2.10	TCP	74 5001 → 46907 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS
5	0.151614	10.0.2.10	10.0.4.10	TCP	66 46907 → 5001 [ACK] Seq=1 Ack=1 Win=14608 Len=0 TSval=12

ISNs intercambiados:



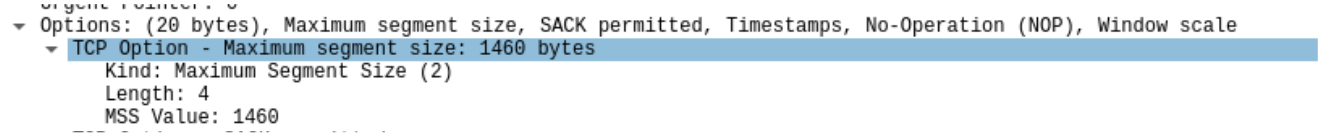
No.	Time	Source	Destination	Protocol	Length	Info
3	0.000079	10.0.2.10	10.0.4.10	TCP	74	46907 → 5001 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
4	0.000116	10.0.4.10	10.0.2.10	TCP	74	5001 → 46907 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS
961	82.420045	10.0.2.10	10.0.4.10	TCP	74	45670 → 7002 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
963	83.540758	10.0.2.10	10.0.4.10	TCP	74	45671 → 7002 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
967	97.968958	10.0.2.10	10.0.4.10	TCP	74	46910 → 5001 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
968	97.969023	10.0.4.10	10.0.2.10	TCP	74	5001 → 46910 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS
981	135.753852	10.0.2.10	10.0.4.10	TCP	74	54424 → 9000 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
982	135.754058	10.0.4.10	10.0.2.10	TCP	74	9000 → 54424 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS
1106	149.807117	10.0.2.10	10.0.4.10	TCP	74	54425 → 9000 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_
1107	149.807136	10.0.4.10	10.0.2.10	TCP	74	9000 → 54425 [SYN, ACK] Seq=0 Ack=1 Win=14480 Len=0 MSS

Utilicé el filtro tcp.flags.syn == 1

- Petición N°3:
  - Seq number: 0
  - Seq number raw: 2218328254
- Petición N°4
  - Seq: 0
  - Seq raw: 1292618479
- Petición N°961
  - Seq: 0
  - Seq raw: 1834854137

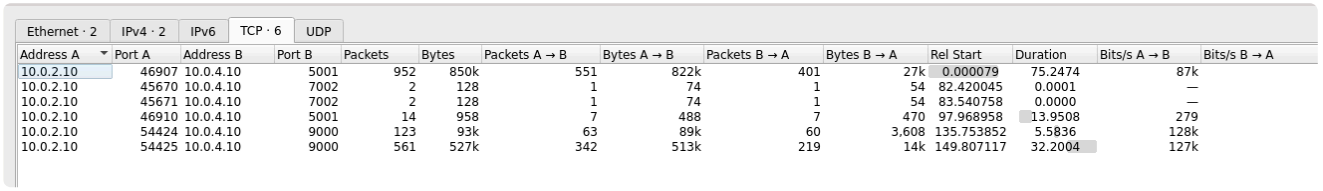
    ...

MSS:



```
Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  TCP Option - Maximum segment size: 1460 bytes
    Kind: Maximum Segment Size (2)
    Length: 4
    MSS Value: 1460
```

El host que envía mayor cantidad de datos es:



Ethernet · 2 | IPv4 · 2 | IPv6 | TCP · 6 | UDP

Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.2.10	46907	10.0.4.10	5001	952	850k	551	822k	401	27k	0.000079	75.2474	87k	
10.0.2.10	45670	10.0.4.10	7002	2	128	1	74	1	54	82.420045	0.0001	—	
10.0.2.10	45671	10.0.4.10	7002	2	128	1	74	1	54	83.540758	0.0000	—	
10.0.2.10	46910	10.0.4.10	5001	14	958	7	488	7	470	97.968958	13.9508	279	
10.0.2.10	54424	10.0.4.10	9000	123	93k	63	89k	60	3,608	135.753852	5.5836	128k	
10.0.2.10	54425	10.0.4.10	9000	561	527k	342	513k	219	14k	149.807117	32.2004	127k	

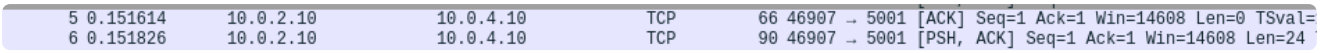
Como bien se visualiza en la imagen, quien envía la mayor cantidad de datos es `10.0.2.10:46907`.

> **Esto se ve en**
>
> **Statistics > Conversations** (Estadísticas > Conversaciones), seleccionar la pestaña **TCP** y contar cuántas líneas o flujos aparecen que

completaron el saludo inicial (*Three-way Handshake*).

Inciso 9.5

El primer segmento de datos real es **el primer paquete que transporta información útil** (es decir, el primero que tiene un `Len` mayor a 0, usualmente marcado con flags como `PSH, ACK`).



- **Tiempo:** `0.151826`
- **Origen:** `10.0.2.10` (Puerto `46907`)
- **Destino:** `10.0.4.10` (Puerto `5001`)
- **Número de secuencia TCP (Seq):** `Seq = 1`

Lleva 24 bytes de datos.



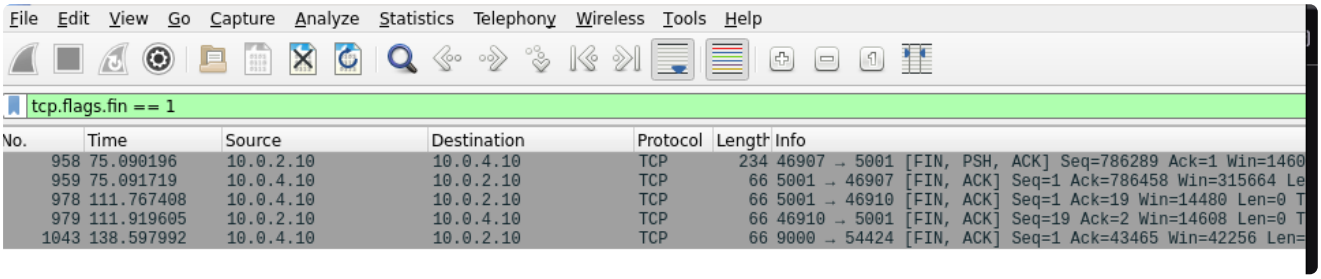
Mirando la fila 7 (la respuesta inmediata que envía `10.0.4.10` a `10.0.2.10`):

- **Tiempo:** `0.151925`
- **Número de fila:** `7`
- **Número de secuencia TCP / ACK:** `Ack=25`

Confirma exactamente **24 bytes**.

TCP utiliza reconocimientos acumulativos donde el campo `Ack` especifica el número de secuencia del *siguiente byte que espera recibir*. Como el paquete de datos comenzó en $\text{Seq} = 1$ y transportó 24 bytes (bytes del 1 al 24), el receptor responde con $\text{Ack} = 25$ ($1 + 24 = 25$), confirmando la recepción exitosa de esos 24 bytes.

Inciso 9.6



El cierre de la conexión principal lo inicia el **cliente (** `10.0.2.10:46907` **)** hacia el **servidor (** `10.0.4.10:5001` **)**. Se ve en la primer línea.
En el inicio del cierre por parte del cliente se utilizan los flags **FIN, PSH, ACK** (el flag **FIN** para indicar el fin de la transmisión, **PSH** para forzar la entrega y **ACK** para confirmar lo previamente recibido).
En la respuesta de cierre del servidor se utilizan los flags **FIN, ACK**.

¿En cuáles segmentos se ve?

Mirando las dos primeras filas de la captura filtrada (`tcp.flags.fin == 1`):

1. **Segmento 1 (Inicio de cierre del cliente):**
   - **Tiempo:** `75.090196`
   - **Número de fila:** `958`
   - **Número de secuencia TCP:** `Seq = 786289`
   - *Flags:* `[FIN, PSH, ACK]`
2. **Segmento 2 (Cierre del servidor):**
   - **Tiempo:** `75.091719`
   - **Número de fila:** `959`
   - **Número de secuencia TCP:** `Seq = 1` (con `Ack = 786458`)
   - *Flags:* `[FIN, ACK]`

Inciso 10

Responda las siguientes preguntas respecto del mecanismo de control de flujo.

1. ¿Quién lo activa? ¿De qué forma lo hace?
2. ¿Qué problema resuelve?
3. ¿Cuánto tiempo dura activo y qué situación lo desactiva?

Inciso 10.1

- **Quién lo activa:** El **receptor** de la conexión TCP.
- **De qué forma lo hace:** El receptor calcula en todo momento el espacio libre que le queda en su buffer de recepción (`BufferRecepcion`) y notifica de forma continua este valor al emisor a través del campo **Ventana de recepción** (*Receive Window* o `VentRecepcion` / `rwnd`) de

la cabecera de los segmentos de retorno. Con esta información, el emisor restringe la cantidad de datos enviados sin confirmar (`UltimoByteEnviado - UltimoByteReconocido <= VentRecepcion`), ajustando dinámicamente su tasa de envío

Inciso 10.2

Resuelve el **desbordamiento del buffer de recepción** (*buffer overflow*) en el host de destino cuando el emisor transmite información a una velocidad superior a la capacidad del proceso receptor para leerla. Es un mecanismo de **adaptación de velocidades** de extremo a extremo que protege la memoria del host receptor. *(A diferencia del control de congestión, que evita saturar la infraestructura de routers y enlaces intermedios de la red, el control de flujo atiende exclusivamente la velocidad del receptor*.

Inciso 10.3

- **Duración:** El mecanismo opera de manera continua **durante todo el ciclo de vida de la conexión TCP**. La situación de **bloqueo o restricción del emisor** se activa en el momento en que el receptor anuncia una ventana de valor cero (`VentRecepcion = 0`) debido a que su buffer se ha llenado. Mientras dura esta condición, el emisor detiene el flujo regular y transmite periódicamente segmentos de sondeo (*zero-window probes*) de 1 byte para consultar si el espacio se ha liberado.
- **Situación que lo desactiva (libera el flujo):** Se desactiva la restricción cuando la aplicación del receptor procesa y lee los datos acumulados, vaciando el buffer. Al liberar espacio, el receptor envía un nuevo segmento con un valor de **ventana de recepción mayor a cero** (`VentRecepcion > 0`), lo que reanuda la transmisión normal por parte del emisor

Inciso 11

Responda las siguientes preguntas respecto del mecanismo de control de congestión.

1. ¿Quién activa el mecanismo de control de congestión? ¿Cuáles son los posibles disparadores?
2. ¿Qué problema resuelve?

3. Diferencie slow start de congestion-avoidance.

Inciso 11.1

- **Quién lo activa:** El **host emisor TCP** ejecuta de forma autónoma el control de congestión de extremo a extremo (*end-to-end*), regulando la velocidad a la que inyecta datos en la red mediante su ventana de congestión (`cwnd` / `VentCongestion`).
- **Posibles disparadores:** El emisor deduce que existe congestión en la red cuando se produce un **evento de pérdida**:
  1. **Expiración del temporizador de retransmisión (timeout):** Ocurre cuando vence el temporizador sin recibir el reconocimiento de un segmento, señalando una congestión severa o descartes en las colas de los routers.
  2. **Recepción de 3 ACKs duplicados (triple ACK):** Dispara la retransmisión rápida (*fast retransmit*) e indica que se perdió un paquete específico, aunque la red conserva cierta capacidad para entregar segmentos posteriores.
  3. **Señalización de la red (en sistemas con ECN):** Si la red utiliza *Notificación Explícita de Congestión* (ECN), la recepción de un segmento ACK con la bandera `ECE` activada funciona como un disparador directo para reducir la tasa de envío.

Inciso 11.2

El control de congestión resuelve la **saturación de la infraestructura de red interconectada** (routers, conmutadores y enlaces) cuando múltiples fuentes transmiten más tráfico del que la red puede procesar.

Específicamente, previene y soluciona:

- **Desbordamiento de buffers en los routers:** Evita que las colas de los conmutadores se colapsen y descarten paquetes masivamente.
- **Retardos excesivos de encolamiento (queuing delays):** Impide que el tiempo de tránsito aumente indefinidamente por colas saturadas.
- **Colapso por congestión:** Evita el desperdicio de ancho de banda provocado por la retransmisión continua de paquetes perdidos o demorados.

- **Reparto equitativo del ancho de banda (fairness):** Garantiza que las conexiones TCP que comparten un enlace obtengan una cuota justa del canal

Inciso 11.3

Criterio	Arranque Lento (Slow Start)	Evitación de la Congestión (Congestion Avoidance)
Objetivo	Sondear rápidamente el ancho de banda disponible al iniciar la conexión o tras un timeout.	Sondear el ancho de banda de forma conservadora y prudente al aproximarse a la capacidad estimada de la red.
Crecimiento de cwnd	Exponencial: Incrementa 1 MSS por cada ACK recibido, lo que duplica la cwnd en cada RTT.	Lineal (Incremento Aditivo / AIMD): Incrementa 1 MSS por cada RTT completo (o MSS * (MSS / cwnd) por cada ACK).
Condición de transición	Transiciona a Congestion Avoidance cuando cwnd alcanza o supera el umbral de arranque lento (ssthresh / umbralAL).	Mantiene el crecimiento lineal hasta detectar una pérdida. Ante un triple ACK divide la cwnd a la mitad, y ante un timeout reinicia la cwnd a 1 MSS volviendo a Slow Start.

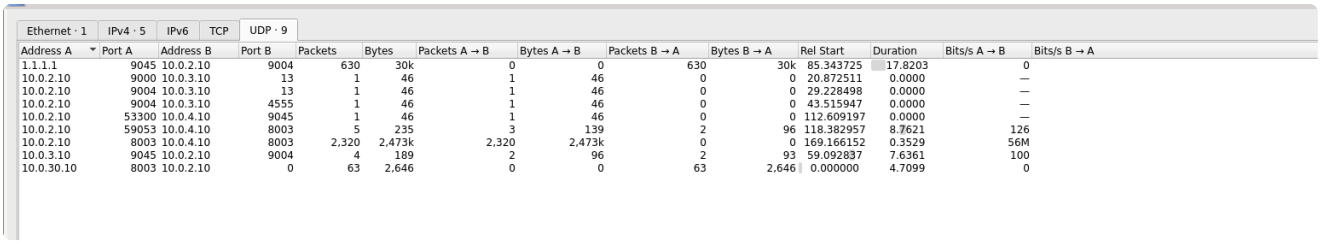
Inciso 12

Para la captura udp-captura.pcap, responder las siguientes preguntas.

1. ¿Cuántas comunicaciones (srcIP,srcPort,dstIP,dstPort) UDP hay en la captura?
2. ¿Cómo se podrían identificar las exitosas de las que no lo son?
3. ¿UDP puede utilizar el modelo cliente/servidor?
4. ¿Qué servicios o aplicaciones suelen utilizar este protocolo?¿Qué requerimientos tienen?
5. ¿Qué hace el protocolo UDP en relación al control de errores?
6. Con respecto a los puertos vistos en las capturas, ¿observa algo particular que lo diferencie de TCP?

7. Dada la primera comunicación en la cual se ven datos en ambos sentidos (identificar el primer datagrama):
    1. ¿Cuál es la dirección IP que envía el primer datagrama?,¿desde cuál puerto?
    2. ¿Cuántos datos se envían en un sentido y en el otro?

Inciso 12.1



Ethernet · 1 | IPv4 · 5 | IPv6 | TCP | UDP · 9

Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
1.1.1.1	9045	10.0.2.10	9004	630	30k	0	0	630	30k	85.343725	17.8203	0	
10.0.2.10	9000	10.0.3.10	13	1	46	1	46	0	0	20.872511	0.0000	—	
10.0.2.10	9004	10.0.3.10	13	1	46	1	46	0	0	29.228498	0.0000	—	
10.0.2.10	9004	10.0.3.10	4555	1	46	1	46	0	0	43.515947	0.0000	—	
10.0.2.10	53300	10.0.4.10	9045	1	46	1	46	0	0	112.609197	0.0000	—	
10.0.2.10	59053	10.0.4.10	8003	5	235	3	139	2	96	118.382957	8.7621	126	
10.0.2.10	8003	10.0.4.10	8003	2,320	2,473k	2,320	2,473k	0	0	169.166152	0.3529	56M	
10.0.3.10	9045	10.0.2.10	9004	4	189	2	96	2	93	59.092837	7.6361	100	
10.0.30.10	8003	10.0.2.10	0	63	2,646	0	0	63	2,646	0.000000	4.7099	0	

De forma rápida, podemos decir que hay 9 conexiones. Cada fila en la ventana de conversaciones representa una 4-upla única `(srcIP, srcPort, dstIP, dstPort)` agrupada bidireccionalmente por el analizador.

Si se analiza el comportamiento de los servicios en destino, varias de estas filas corresponden a la **misma comunicación/servicio**. A diferencia de TCP, en UDP la demultiplexación no exige una conexión previa ni evalúa la 4-upla completa para entregar el datagrama; el proceso receptor atiende únicamente en base a **(IP destino, Puerto destino)**:

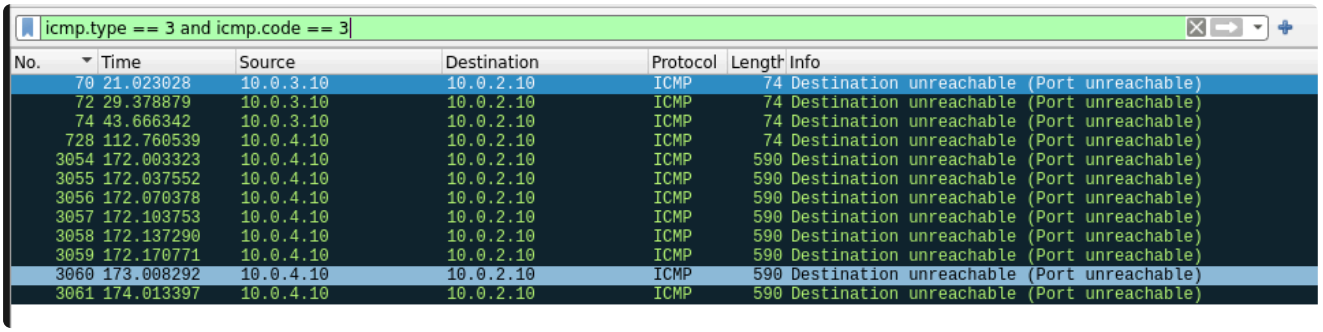
- Las **filas 2 y 3** van dirigidas al mismo socket servidor (`10.0.3.10:13` , Daytime).
- Las **filas 6 y 7** van dirigidas al mismo proceso en `10.0.4.10:8003` .
- Las **filas 1 y 8** corresponden al mismo socket local (`10.0.2.10:9004`) interactuando con el puerto remoto `9045` .

Inciso 12.2

> **UDP**
>
> Cuando un datagrama UDP llega a un host de destino donde no hay ningún proceso escuchando en ese puerto (puerto cerrado), el sistema operativo devuelve un error de capa de red: **ICMP Destination**

Unreachable (Port Unreachable - Tipo 3, Código 3)

Filtro: icmp.type == 3 and icmp.code == 3



Inciso 12.3

UDP puede utilizar y utiliza habitualmente el modelo **cliente/servidor** en la capa de aplicación.

1. **El Servidor UDP:**
   - Crea un socket UDP y lo vincula (*bind*) a un número de puerto de destino bien conocido (por ejemplo, el puerto 53 para DNS o el 67 para DHCP).
   - Permanece a la espera de datagramas entrantes provenientes de cualquier cliente mediante llamadas como `recvfrom()`.
   - **Diferencia clave con TCP:** A diferencia de TCP (que necesita un socket de escucha y crear un nuevo socket por cada cliente), el servidor UDP **solo necesita un único socket** para atender a múltiples clientes simultáneamente.
2. **El Cliente UDP:**
   - Crea su propio socket UDP, al cual el sistema operativo le asigna de forma transparente un puerto de origen efímero.
   - Envía el datagrama directamente al servidor indicando su dirección IP y puerto de destino mediante `sendto()`.
3. **Respuesta y Dirección de Retorno:**
   - Cada datagrama UDP incluye en sus encabezados la IP y el puerto de origen del emisor.
   - El servidor extrae esta información y la utiliza como **dirección de retorno** para enviarle la respuesta al cliente si la aplicación lo requiere.

Ejemplos clásicos de protocolos Cliente/Servidor sobre UDP

- **DNS (Domain Name System):** El cliente (*resolver*) envía consultas en datagramas al puerto 53 UDP del servidor de nombres.
- **DHCP (Dynamic Host Configuration Protocol):** El cliente solicita una dirección IP al servidor DHCP sobre los puertos 67 y 68 UDP.
- **NTP, TFTP y SNMP:** Protocolos de sincronización horaria, transferencia liviana de archivos y gestión de red.

Inciso 12.4

1. **Servicios y aplicaciones que utilizan UDP**
   - **Traducción de nombres de dominio (DNS):** Consultas de búsqueda e interacciones breves de pregunta-respuesta que requieren respuestas prácticamente instantáneas.
   - **Servicios de configuración y administración de red: DHCP / BOOTP** (asignación dinámica de IP), **NTP** (sincronización horaria), **SNMP** (gestión de red) y **TFTP** (transferencia trivial de archivos).
   - **Aplicaciones multimedia en tiempo real y VoIP:** Telefonía IP (Skype), videoconferencias y streaming de audio/vídeo (usando protocolos auxiliares como **RTP**).
   - **Protocolos modernos sobre UDP (QUIC / HTTP/3):** Implementaciones como **QUIC** (usado por navegadores como Chrome) ejecutan la transferencia web sobre UDP, implementando la fiabilidad y el cifrado directamente en la capa de aplicación.
   - **Juegos interactivos en red:** Aplicaciones donde el estado del juego se actualiza constantemente y los datos antiguos pierden valor rápidamente.
2. **Requerimientos que tienen:**
   1. **Sensibilidad al tiempo y baja latencia (Time-sensitive):**
      - Requieren que los datos lleguen sin el retardo adicional que provocaría el saludo en tres fases (*three-way handshake*) o las retransmisiones de paquetes perdidos propias de TCP.
      - En voz interactiva (VoIP), por ejemplo, un retardo superior a 150–400 ms vuelve la conversación incómoda o ininteligible.

2. **Tolerancia a pérdidas de datos (Loss-tolerant):**
    - A diferencia del correo o la web (donde la pérdida de un bit corrompe el archivo), estas aplicaciones pueden tolerar pequeñas pérdidas ocasionales de paquetes.
    - En voz o vídeo, perder un datagrama solo genera un chasquido casi imperceptible o un salto de fotograma.
3. **Inexistencia de control de congestión en la capa de transporte:**
    - Las aplicaciones de voz/vídeo necesitan emitir a una **tasa constante o mínima garantizada**.
    - El control de congestión de TCP reduciría drásticamente la tasa de envío ante pérdidas en la red, lo que provocaría la "inanición" (*starvation*) del búfer de reproducción.
4. **Baja sobrecarga en la cabecera (Low overhead):**
    - UDP utiliza una cabecera fija de solo **8 bytes** (frente a los **20 bytes** mínimos de TCP), lo cual ahorra ancho de banda en transmisiones frecuentes de paquetes pequeños.
5. **Fiabilidad controlada por la aplicación (opcional):**
    - Si la aplicación requiere alguna garantía (como detectar orden con números de secuencia o recuperar paquetes), prefiere incorporar esa lógica en la **capa de aplicación** (mediante **RTP** o **QUIC**) sin quedar atada a las rigideces de TCP.

Inciso 12.5

UDP realiza únicamente detección de errores, pero no proporciona ningún mecanismo de recuperación.

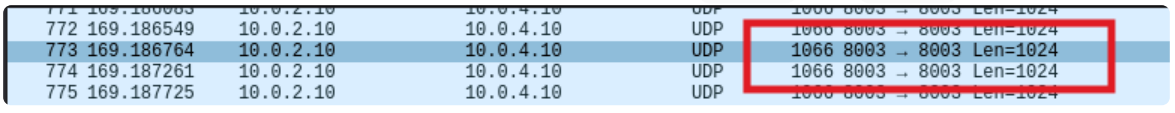
Su funcionamiento frente a los errores se compone de los siguientes puntos:

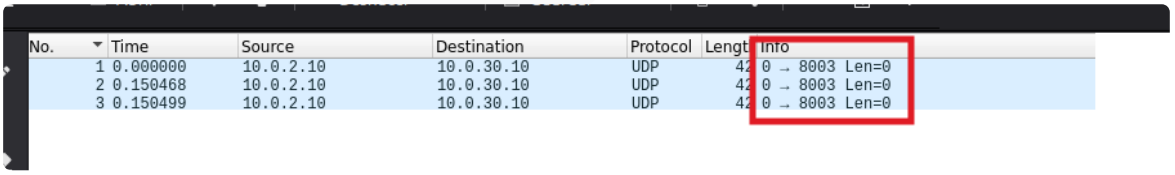
1. Detección de errores mediante Checksum
    - **Campo Checksum:** UDP incluye en su cabecera de 8 bytes un campo de **suma de comprobación (checksum)** de 16 bits
    - **Cálculo en el emisor:** El emisor trata los contenidos del segmento UDP, la cabecera y una pseudo-cabecera IP (de 12 bytes) como una secuencia de enteros de 16 bits, calcula la suma en complemento a 1 y guarda el resultado en el campo *checksum*.

- **Verificación en el receptor:** El host receptor realiza el cálculo sobre el segmento recibido. Si la suma no resulta en todos 1s en binario, se detecta que los bits fueron alterados en el tránsito.
- **Diferencia entre IPv4 e IPv6:** En IPv4 el uso del *checksum* en UDP es opcional (se puede deshabilitar poniendo el campo en cero), mientras que en IPv6 es obligatorio.

2. Ausencia de recuperación de errores
   - **Sin retransmisión ni confirmaciones:** UDP es un protocolo no orientado a conexión que carece de acuses de recibo (ACKs/NAKs), números de secuencia, temporizadores de retransmisión o control de flujo.
   - **Acción ante un paquete dañado:** Cuando la comprobación de *checksum* falla, UDP **no solicita la retransmisión del paquete**. La gran mayoría de las implementaciones simplemente **descartan el segmento corrompido en silencio** (o, en algunos casos aislados, lo entregan a la aplicación adjuntando una advertencia).
3. Delegación a la capa de aplicación
   - Si una aplicación transmitida sobre UDP necesita garantizar la entrega sin errores ni pérdidas, **debe implementar su propia lógica de fiabilidad en la capa de aplicación** (incorporando sus propios números de secuencia, confirmaciones y retransmisiones, como hace por ejemplo el protocolo **QUIC**).

Inciso 12.6

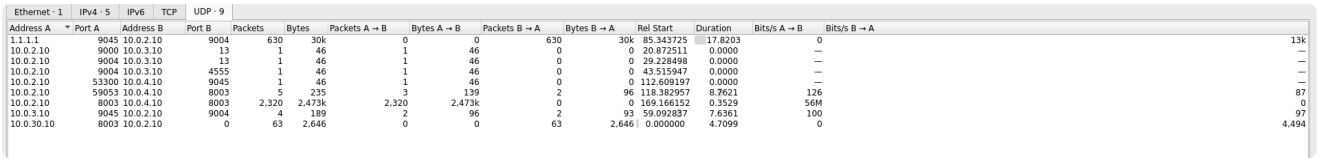
- Mismo puerto de origen y destino simultáneo (`8003` → `8003`)
  - 

  - Cabe aclarar que en TCP es teóricamente posible según el estándar (RFC 793), pero casi nunca se ve en la práctica cotidiana.
- Puerto de origen igual a 0 (`Port A = 0`)
  - 

- Reutilización del mismo puerto local para distintos destinos y servicios sin conexión

- El socket local `10.0.2.10:9004` aparece enviando tráfico a diferentes destinos y servicios (`1.1.1.1:9045` , `10.0.3.10:13` y `10.0.3.10:4555`). En TCP, cada conexión queda estrictamente acoplada a una **4-upla** fija `(IP origen, Puerto origen, IP destino, Puerto destino)` respaldada por un socket de conexión exclusivo (`connectionSocket`). En cambio, en UDP un único socket datagrama local puede ejecutar múltiples llamadas a `sendto()` y disparar paquetes hacia IPs y puertos totalmente distintos sin mantener estado ni requerir un socket por cada extremo.

Inciso 12.7



Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
1.1.1.1	9045	10.0.2.10	9004	630	30k	0	0	630	30k	85.343725	17.8203	0	13k
10.0.2.10	9000	10.0.3.10	13	1	46	1	46	0	0	20.872511	0.0000	—	—
10.0.2.10	9004	10.0.3.10	13	1	46	1	46	0	0	29.228498	0.0000	—	—
10.0.2.10	9004	10.0.3.10	4555	1	46	1	46	0	0	43.515947	0.0000	—	—
10.0.2.10	53300	10.0.4.10	9045	1	46	1	46	0	0	112.609197	0.0000	—	—
10.0.2.10	59053	10.0.4.10	8003	5	235	3	139	2	96	118.382957	8.7621	126	87
10.0.2.10	8003	10.0.4.10	8003	2,320	2,473k	2,320	2,473k	0	0	169.166152	0.3529	56M	0
10.0.3.10	9045	10.0.2.10	9004	4	189	2	96	2	93	59.092837	7.6361	100	97
10.0.30.10	8003	10.0.2.10	0	63	2,646	0	0	63	2,646	0.000000	4.7099	0	4,494

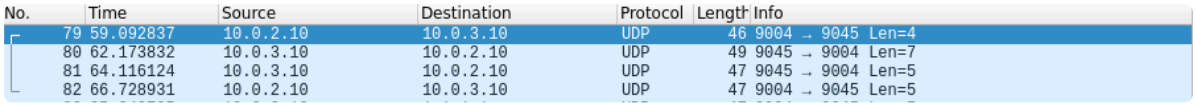
Debemos busca las filas que tengan tráfico en ambos sentidos (`Packets A -> B > 0` y `Packets B -> A > 0`).

- La fila 6 muestra: `10.0.2.10:59053` , `10.0.4.10:8003` , con inicio en el segundo `118.382957` .
- La fila 8 muestra: `10.0.3.10:9045` , `10.0.2.10:9004` , con inicio en el segundo `59.092837` .
- Comparando los tiempos de inicio (`Rel Start`), **la primera en ocurrir en el tiempo es la de la fila 8** ($59.09\ \text{s}$ es anterior a $118.38\ \text{s}$).

Filtro a usar en wireshark:

- udp.port == 9004 and udp.port == 9045

1. La dirección ip que envía el primer datagrama



No.	Time	Source	Destination	Protocol	Length	Info
79	59.092837	10.0.2.10	10.0.3.10	UDP	46	9004 → 9045 Len=4
80	62.173832	10.0.3.10	10.0.2.10	UDP	49	9045 → 9004 Len=7
81	64.116124	10.0.3.10	10.0.2.10	UDP	47	9045 → 9004 Len=5
82	66.728931	10.0.2.10	10.0.3.10	UDP	47	9004 → 9045 Len=5

   - **Dirección IP de origen:** `10.0.2.10`
   - **Puerto de origen:** `9004`
   - Enviado hacia el destino: `10.0.3.10:9045` , con `Len=4` .
2. ¿Cuántos datos se envían en un sentido y en el otro?
   - **En el sentido de ida (`10.0.2.10:9004` → `10.0.3.10:9045`):**

    - Corresponde a la fila 79, que transporta `4 bytes` de datos de aplicación (`Len=4`). Longitud total a nivel de red/trama: 46 bytes.
- **En el sentido de vuelta / respuesta (`10.0.3.10:9045` → `10.0.2.10:9004`):**
    - Corresponde a la fila 80, que devuelve `7 bytes` de datos de aplicación (`Len=7`). Longitud total a nivel de red/trama: 49 bytes.